

Name:P.Jeeva

Reg No:192412636

Course Code:ITA0609

CO2-AT2-DEBUGGING TASK

1. Debugging K-Means Clustering Algorithm Producing Inconsistent Cluster Assignments

Problem:

A K-means clustering algorithm gives different cluster assignments even when similar datasets are used. This indicates instability in the clustering process.

Possible Reasons:

1. Poor Initialization of Centroids

- K-means randomly selects initial cluster centers.
- Different starting points may produce different final clusters.

2. Incorrect Number of Clusters (K Value)

- Selecting an unsuitable value of K affects clustering accuracy.
- A small K may merge different groups, while a large K may divide similar groups unnecessarily.

3. Sensitivity to Outliers

- K-means calculates cluster centers using the mean value.
- Extreme data points can shift centroids and create incorrect clusters.

4. Feature Scaling Problems

- Features with larger numerical values dominate distance calculations.
- This results in biased cluster formation.

Debugging Approaches:

- Visualize the dataset using scatter plots.
- Test different K values and compare clustering results.
- Identify and remove abnormal/outlier data points.
- Check feature ranges and distributions.

Optimization Strategies:

1. K-Means++ Initialization

- Selects better initial centroids instead of random selection.
- Improves stability and reduces variation between runs.

2. Normalization/Standardization

- Converts features into a common scale.
- Ensures all features contribute equally.

3. Silhouette Analysis

- Evaluates how well data points fit within their clusters.
- Helps determine the optimal number of clusters.

4. Multiple Runs

- Execute K-means several times and choose the best clustering result.

Conclusion:

Using proper initialization, feature scaling, and cluster evaluation techniques improves the reliability and consistency of K-means clustering.

2. Debugging SVM Model Underperforming on Nonlinear Dataset

Problem:

A Support Vector Machine (SVM) model performs poorly on a nonlinear dataset because it fails to create an appropriate decision boundary.

Possible Reasons:

1. Incorrect Kernel Selection

- A linear kernel cannot classify complex nonlinear patterns.
- The model requires a nonlinear kernel such as RBF or Polynomial.

2. Improper Hyperparameter Tuning

- Incorrect values of parameters like C and gamma reduce performance.
- Low C may cause underfitting, while high C may cause overfitting.

3. Feature Scaling Issues

- SVM uses distance calculations.
- Features with different scales can affect the decision boundary.

4. Irrelevant Features

- Unnecessary features may reduce classification accuracy.

Debugging Techniques:

- Compare training and testing accuracy to identify overfitting or underfitting.
- Analyze classification errors using a confusion matrix.
- Check feature distributions and scaling.
- Perform cross-validation to evaluate model performance.

Optimization Methods:

1. Kernel Selection

- Use RBF kernel for complex nonlinear datasets.
- Use Polynomial kernel for curved decision boundaries.

- Select kernel based on dataset characteristics.

2. Hyperparameter Tuning

- Use Grid Search or Random Search.
- Optimize parameters such as:
 - C (regularization parameter)
 - Gamma (kernel influence)
 - Kernel type

3. Feature Scaling

- Apply normalization or standardization before training.
- Improves distance calculation and model accuracy.

4. Feature Selection

- Remove unnecessary features to improve efficiency.

Conclusion:

Proper kernel selection, parameter tuning, and feature scaling help SVM achieve better performance on nonlinear datasets.

3. Debugging RNN Model with Poor Long-Term Dependency Learning in NLP

Problem:

A Recurrent Neural Network (RNN) used for NLP fails to remember important information from earlier parts of a sequence.

Possible Reasons:

1. Vanishing Gradient Problem

- During backpropagation, gradients become very small.
- The network cannot learn long-term relationships between words.

2. Limited Memory Capacity

- Basic RNNs store information only for a short time.
- Important context from previous words may be lost.

3. Insufficient Training Strategy

- Poor learning rate, insufficient data, or improper training can reduce performance.

Debugging Steps:

- Monitor training and validation loss curves.
- Check gradient values during training.
- Analyze sequence length and dataset quality.
- Compare performance with different model architectures.

Solutions and Optimization Strategies:

1. LSTM (Long Short-Term Memory) Units

- Uses memory cells and gates to store important information.
- Solves the vanishing gradient problem.

2. GRU (Gated Recurrent Unit)

- A simpler alternative to LSTM.
- Provides better memory handling with fewer parameters.

3. Attention Mechanism

- Allows the model to focus on important words in a sequence.
- Improves understanding of long-range dependencies.

4. Improved Training Strategies

- Use proper learning rates.
- Apply dropout to reduce overfitting.
- Use larger and better-quality datasets.
- Apply batch normalization and advanced optimizers.

Conclusion:

Replacing basic RNNs with LSTM/GRU models and using attention mechanisms improves long-term dependency learning and NLP performance.